DATA CLEANING LOG

Customer Transaction Dataset

Field	Detail
Document Type	Data Cleaning Log
Dataset	Customer Transaction Dataset
Source	Google Drive (CSV)
Notebook File	02_Cleaning.ipynb
Tools Used	Python, Pandas, NumPy
Purpose	Clean raw transaction data for analysis
Status	Completed

1. Project Overview

This document is a structured log of all data cleaning steps performed on a raw customer transaction dataset. The dataset was loaded from a remote CSV file and processed using Python (Pandas). The goal was to produce a clean, analysis-ready dataset by handling duplicates, fixing data types, and imputing missing values logically.

2. Step 1 — Library Imports & Setup

The following Python libraries were imported to support data loading, manipulation, and visualization:

Library	Purpose
pandas	Core data manipulation and cleaning
numpy	Numerical operations and array handling
matplotlib.pyplot	Static data visualizations
seaborn	Statistical plots and styling
plotly.express	Interactive visualizations

Additionally, display settings were adjusted using pd.set_option('display.max_columns', None) to ensure all columns were visible during exploratory inspection — without this, wide DataFrames truncate columns in the output.

3. Step 2 — Data Gathering

The dataset was loaded directly from a Google Drive URL using pd.read_csv():

```
df = pd.read_csv("https://drive.google.com/uc?id=1q5OsZDzQWmie0kp5SwXFN8NyJ_qkJf7T")
```

Initial exploration was performed using the following commands:

- df.shape — to check the number of rows and columns
- df.head() — to preview the first 5 rows
- df.tail() — to preview the last 5 rows
- df.info() — to inspect data types, column names, and non-null counts

4. Step 3 — Drop Unwanted Columns

Several columns were identified as personally identifiable information (PII) that are irrelevant for analysis. These were dropped to reduce noise and protect data privacy.

Columns Dropped:

Column	Reason for Removal
Name	PII — not required for analysis
Email	PII — not required for analysis
Phone	PII — not required for analysis
Address	PII — not required for analysis
Zipcode	PII — not required for analysis

The original shape was recorded before the drop. After removal, the shape was verified and a summary printed showing columns removed and remaining column list.

5. Step 4 — Handle Duplicates

Duplicate detection and removal was done in two levels to ensure data integrity:

Level 1 — Full Duplicate Rows

Rows that were completely identical across all columns were identified and removed using drop_duplicates(keep='first'). The first occurrence was retained. Shape was logged before and after.

Level 2 — Duplicate Transaction IDs

Even after removing fully identical rows, some Transaction_IDs may still be duplicated (e.g., same ID with slightly different values — a data entry error). These were removed with:

```
df.drop_duplicates(subset=['Transaction_ID'], keep='first', inplace=True)
```

A final summary was printed showing the total rows removed across both levels and the final shape of the DataFrame.

6. Step 5 — Data Type Fixing

Correct data types are essential for accurate operations (e.g., arithmetic on amounts, date-based filtering). Each column group was fixed systematically.

Column(s)	Original Type	Fixed Type	Notes
Transaction_ID, Customer_ID	object/int	Int64	Nullable integer to handle NaN values
Date	object	datetime64	Converted with pd.to_datetime()
Year	int	Int64	Extracted from Date column
Month	object	Ordered Categorical	Ordered Jan–Dec for time-series sorting
Time	object	time	Parsed with format '%H:%M:%S'
Age, Total_Purchases, Ratings	float/object	Int64	Converted to integer
Amount, Total_Amount	float	float (rounded 2dp)	Rounded to 2 decimal places
City, State, Country, Gender, etc.	object	category	14 categorical columns converted

Note: Int64 (capital I) was used instead of int64 to support pandas nullable integers, which handle NaN values without converting the column to float.

7. Step 6 — Handle Missing Values

Missing values were handled differently depending on the nature of each column — dropping was reserved for critical ID/amount columns, while imputation was used for others.

6.1 — Drop Critical Rows

Rows missing values in key anchor columns were dropped entirely, as these rows cannot be reliably imputed:

- Transaction_ID — Primary key; cannot be guessed
- Customer_ID — Foreign key; cannot be guessed
- Amount — Core financial field; cannot be assumed

```
df = df.dropna(subset=['Transaction_ID', 'Customer_ID', 'Amount'])
```

6.2 — Numerical Imputation

For numeric columns Age and Ratings, missing values were filled with the column median. The median is preferred over the mean because it is robust to outliers in skewed distributions.

```
df[col].fillna(df[col].median(), inplace=True)
```

6.3 — Categorical Imputation

For 12 categorical columns (Gender, City, State, Country, Customer_Segment, Product_Category, Product_Brand, Feedback, Shipping_Method, Payment_Method, Order_Status, Income), missing values were filled with the column mode (most frequently occurring value).

```
df[col].fillna(df[col].mode()[0], inplace=True)
```

6.4 — Smart Imputation: Financial Columns

Missing values in Total_Purchases, Amount, and Total_Amount were handled using the mathematical relationship between them:

$$\texttt{Total\_Amount} = \texttt{Amount} \times \texttt{Total\_Purchases}$$

Case	Logic Applied
Total_Purchases is missing	Total_Purchases = Total_Amount / Amount
Amount is missing	Amount = Total_Amount / Total_Purchases
Both are missing	Fill with column median as fallback
Final consistency fix	Recalculate Total_Amount = Amount × Total_Purchases (rounded to 2dp)

This approach ensures logical consistency across the three financial columns rather than blindly imputing random values. The final consistency fix ensures no mismatch remains between Amount, Total_Purchases, and Total_Amount.

6.5 — Fix DateTime Columns

After imputation, Date rows with NaN were dropped. Year and Month were re-extracted from the cleaned Date column. Missing Time values were filled with '00:00:00' as a safe default placeholder.

8. Step 7 — Validate & Verify

After all cleaning steps, a final validation pass was performed to confirm data quality:

Check	Method	Expected Result
Null values	df.isnull().sum()	0 nulls in all columns
Shape confirmation	df.shape	Reduced rows, same (cleaned) columns
Data types	df.dtypes	All columns match expected types
Duplicate rows	df.duplicated().sum()	0 duplicates
Full info summary	df.info()	Non-null counts match total rows

All checks passed after the cleaning process. The dataset was confirmed clean and consistent.

9. Step 8 — Export Cleaned Dataset

The final cleaned DataFrame was exported to a CSV file for downstream use:

```
df.to_csv('cleaned_dataset.csv', index=False)
```

The index=False parameter ensures row indices are not written as a column in the exported file.

10. Cleaning Summary

Phase	Action	Outcome
Setup	Imported libraries, set display options	Environment ready
Data Load	Loaded CSV from Google Drive URL	Raw DataFrame created
Column Drop	Removed 5 PII columns	Dataset anonymized

Phase	Action	Outcome
Duplicates	Removed full duplicates + duplicate Transaction IDs	Unique rows retained
Data Types	Fixed 20+ columns across IDs, dates, numeric, categorical	All types corrected
Missing Values	Dropped critical NaNs; imputed numeric, categorical, financial cols	Zero nulls remaining
Validation	Null check, shape check, dtype check, duplicate check	All checks passed
Export	Saved to cleaned_dataset.csv	Analysis-ready file produced

End of Data Cleaning Log